OPERATING SYSTEM

PRACTICAL – EXERCISE

1. Create a new process by invoking the appropriate system call. Get the process identifier of the currently running process and its respective parent using system calls and display the same using a C program.

```
#include <stdio.h>
#include <windows.h>

int main() {
  STARTUPINFOA si = {0};
  PROCESS_INFORMATION pi = {0};

  si.cb = sizeof(si);

  printf("Parent Process\n");
  printf("Process ID (PID): %lu\n", GetCurrentProcessId());

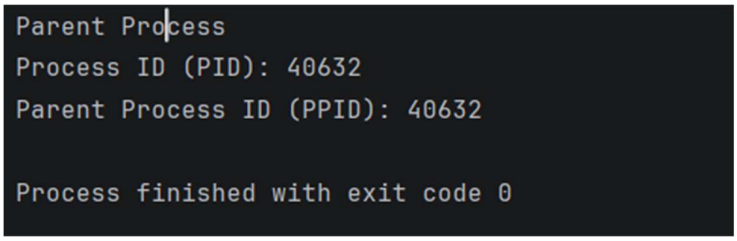
  if (CreateProcessA(
    "C:\\Windows\\System32\\notepad.exe",
    NULL, NULL, NULL, FALSE, 0,
    NULL, NULL, &si, &pi)) {


    printf("Parent Process ID (PPID): %lu\n",
        GetCurrentProcessId());

    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
    } else {
      printf("Process creation failed: %lu\n", GetLastError());
    }

  return 0;
}
```

OUTPUT



2. Create a new process by invoking the appropriate system call. Get the process identifier of the

currently running process and its respective parent using system calls and display the same

using a C program.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
  FILE *fptr1, *fptr2;
  char filename[100], c;
  printf("Enter the filename to open for reading \n");
  scanf("%s", filename);
  fptr1 = fopen(filename, "r");
  if (fptr1 == NULL)
  {
    printf("Cannot open file %s \n", filename);
    exit(0);
  }
  printf("Enter the filename to open for writing \n");
  scanf("%s", filename);
  fptr2 = fopen(filename, "w");
  if(fptr2 == NULL)
  {
    printf("Cannot open file %s \n", filename);
    exit(0);
  }
  c = fgetc(fptr1);
```

```
  while (c != EOF)
  {
    fputc(c, fptr2);
    c = fgetc(fptr1);
  }
  printf("\nContents copied to %s", filename);
  fclose(fptr1);
  fclose(fptr2);
  return 0;
}
```

OUTPUT

